

WEB BASED CHAT APPLICATION

Submitted as a partial fulfillment of Software Lab (CS699)

of

IIT BOMBAY



Project Report

Submitted by

Name of Students

Chiranmoy Bhattacharya

Niraj Jaiswal

Roll No.

22M0744

22M0779

Introduction

Chat application is nothing but a way of real-time communication among different users who may or may not be present at same location. Our Project implements a web-based chat application which is effective for this type of communication.

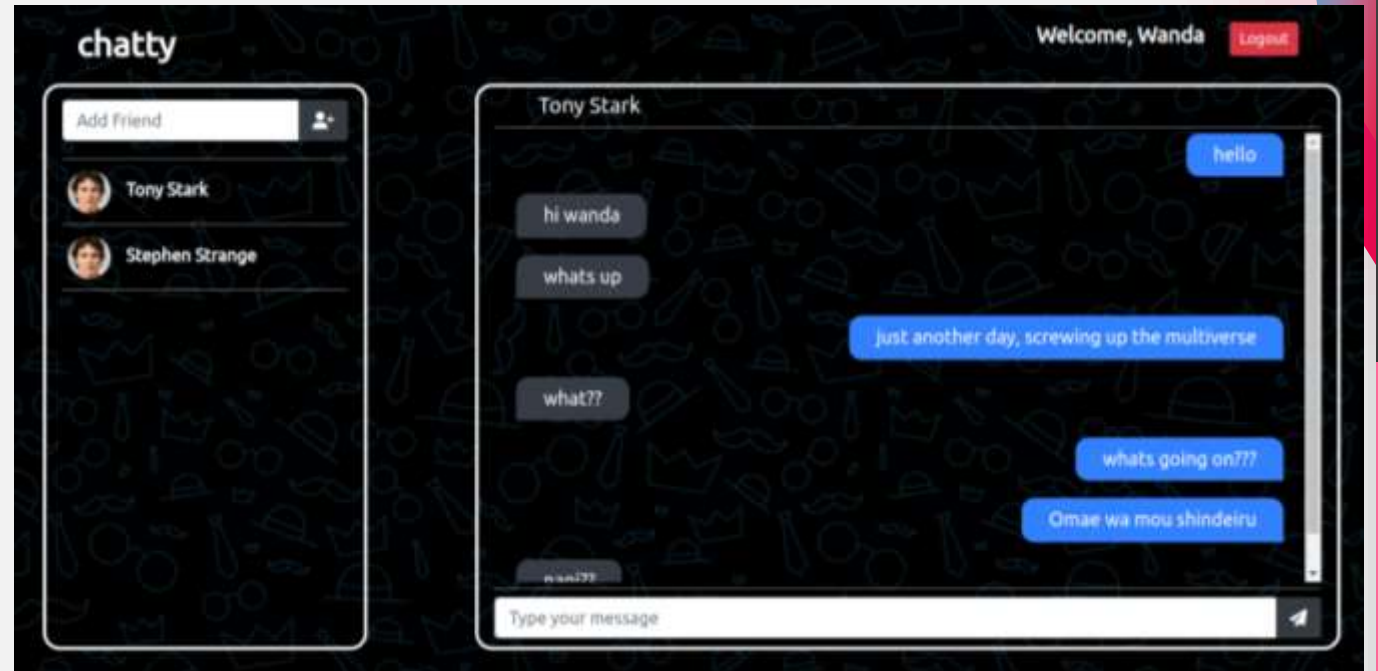


Figure-1 : Chat Page



■ Features —

- Signup and login
- Create a friend list
- Real-time chat



■ Sign-up :

Fig. 2 shows the sign-up page of the chat-app. Before using the chat application, user needs to register in the application by entering the necessary details like phone number, name, email, etc. The user cannot register twice using the same credentials. This functionality is implemented in our project. Once the user has registered, he/she cannot log into the app using the same credentials entered before for signing up in the app. Data collected using this page is stored in the postgresql database.

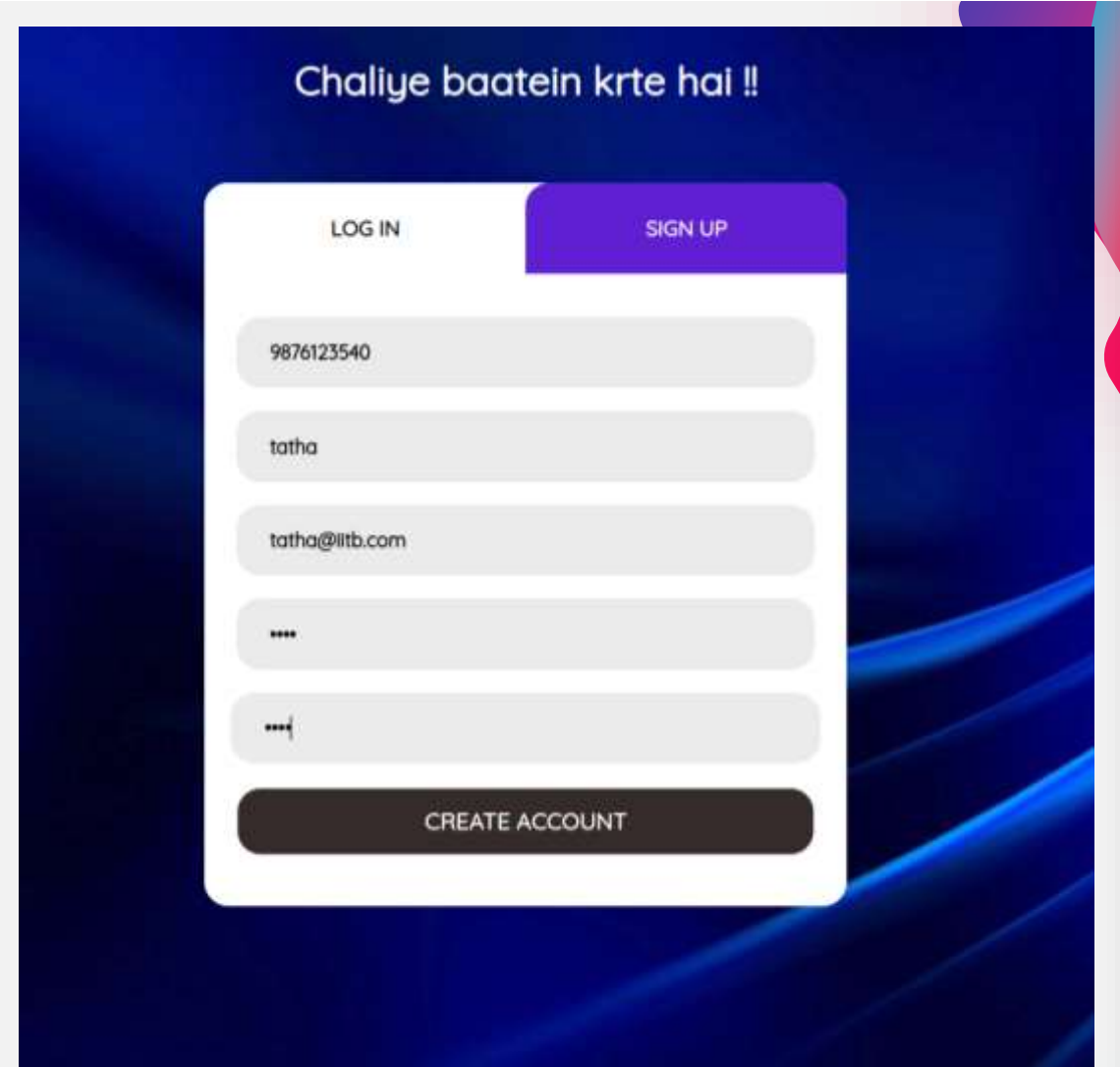
The image shows a web-based sign-up interface for a chat application. At the top, a blue banner contains the text "Chaliye baatein krte hai !!". Below this, a white card contains the login and sign-up options. The "SIGN UP" button is highlighted in purple. The form fields include a phone number (9876123540), a name (tatha), an email (tatha@iitb.com), and two password fields (both masked with dots). A "CREATE ACCOUNT" button is located at the bottom of the form.

Figure-2 : Signup Page



■ Login –

Fig. 3 shows the login page of the chat application. Once the user has registered in the chat application, he/she can log into the page using the same user credentials. All the information entered in the sign-up page is stored in postgresql database. Using the data stored in the database we authenticate the user and then the user is allowed to chat.

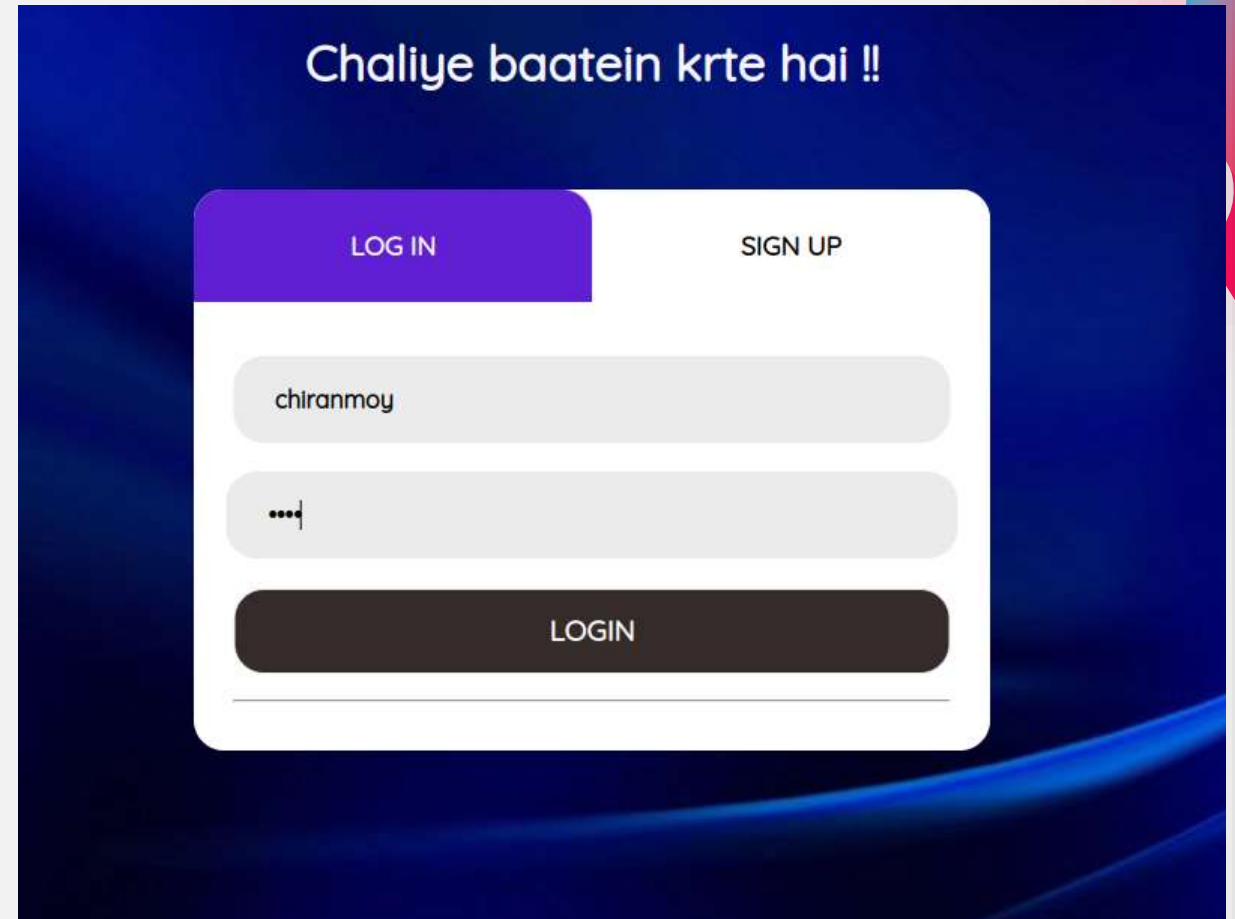


Figure-3 : Login Page



■ Friend List –

Fig. 4 shows the code snippet to add friends in the chat application. We are taking the username as input and then finding whether the data of this user is present in database or not. If the data is present, username of this friend is added to this list.

```
def add_friend(request):  
    if request.method == 'POST':  
        friend = request.POST['friend']  
        me = request.user.username  
  
        if User.objects.filter(username=friend).exists() and friend != me:  
            if not Connections.objects.filter(me=me, friend=friend).exists():  
                Connections.objects.create(me=me, friend=friend)  
                Connections.objects.create(me=friend, friend=me)  
  
    return HttpResponse('')
```

Figure-4

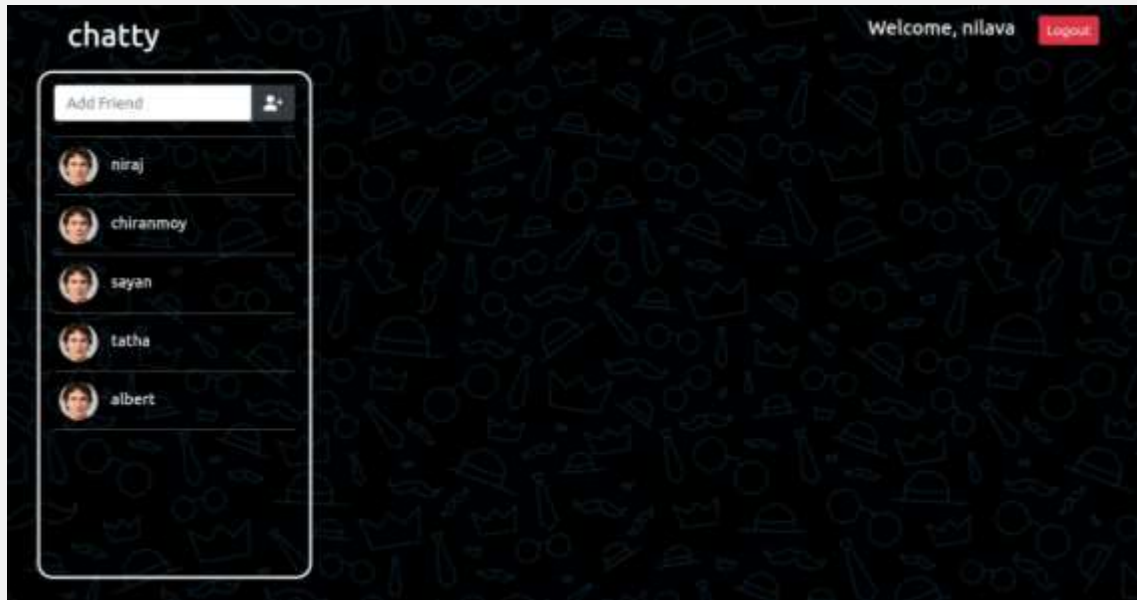


Figure-5 : friend list after adding friends

Fig. 5 shows the updated list of friends i.e., after adding the registered users. We can chat with any of these added friends.



■ Real-time chat –

The real-time chat is performed with the help of web sockets in the front-end and asynchronous channels in the backend, which is done using Django. In the front-end we have used the websockets functionality of javascript. In Django we have used asynchronous sever gateway interface for this real-time chatting. The code snippet used is shown in fig. 6 and 7.

```
var socket = new WebSocket(endpoint)

socket.onopen = async function (e) {
  console.log('open', e)
  let message = document.getElementById('message')
  let submit_btn = document.getElementById('submit_btn')
  let sender = document.getElementById('sender')

  $(document).on('submit', '#message-form', function (e) {
    e.preventDefault();
    console.log("HI FROM NEW")
    let receiver = document.getElementById('receiver')

    let data = {
      'message': message.value,
      'sender': sender.value,
      'receiver': receiver.value
    }

    data = JSON.stringify(data)
    socket.send(data)
    document.getElementById('message-form').reset()
  })
}

socket.onmessage = async function (e) {
  console.log('Message Received', e)
```

Figure-6

```
class ChatappConsumer(AsyncConsumer):
    async def websocket_connect(self, event):
        user = self.scope['user']
        print('*****[CONNECTED]*****', user)
        chatroom = f'user_chatroom_{user.username}'
        self.chatroom = chatroom

        await self.channel_layer.group_add(
            chatroom,
            self.channel_name
        )

        await self.send({
            'type': 'websocket.accept'
        })

    async def websocket_receive(self, event):
        received_msg = json.loads(event['text'])
        message = received_msg.get('message')
        sender = received_msg.get('sender')
        receiver = received_msg.get('receiver')

        print(f"*****[MESSAGE RCVD]*****\n\
            SENDER: {sender}; RECEIVER: {receiver}; MESSAGE: {message}")

        if not message:
            print("*****[ERROR] Empty Message")
```

Figure-7



Learnings

- HTML, CSS, and Bootstrap for Front-end of the application
- How javascript websockets are connected with Django channels
- Backend using Django, asynchronous communication using channels in the Django and websockets
- To save and use data using database i.e., postgresql
- To integrate the complete environment i.e., front-end and backend alongwith database



Future work

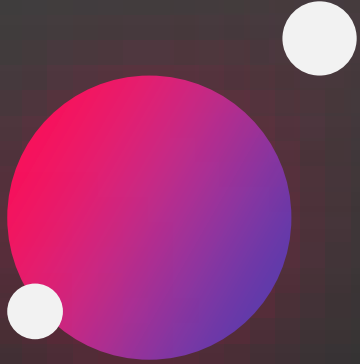
- Last login, read and delivery receipts
- Add support for group chat
- In-app and Push notifications in desktop
- Audio rooms
- Chatbots to provide automated functionalities



References

- <https://getbootstrap.com/docs/4.0/getting-started/introduction/>
- <https://docs.djangoproject.com/en/4.1/>
- https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Positioning
- <https://channels.readthedocs.io/en/stable/asgi.html>
- https://developer.mozilla.org/en-US/docs/Web/API/WebSocket/message_event
- <https://javascript.info/websocket>
- <https://docs.djangoproject.com/en/4.1/topics/db/queries/>





Thank You